BDVectorial_yeisonpd

Documentación Técnica: Arquitectura RAG, Integración con n8n y Frontend Web Chat

1. ¿QUÉ ES UNA BASE DE DATOS VECTORIAL?

Una base de datos vectorial es un sistema de almacenamiento diseñado para guardar, gestionar y recuperar información en forma de **embeddings** (vectores numéricos de alta dimensión). A diferencia de las bases de datos relacionales tradicionales que buscan coincidencias exactas mediante SQL, las bases de datos vectoriales calculan la **similitud semántica** (mediante métricas como la distancia del coseno o la distancia euclidiana) para encontrar conceptos conceptualmente similares, aunque no compartan las mismas palabras exactas.

2. PRINCIPALES TIPOS DE BASES DE DATOS VECTORIALES

1. Nativas Especializadas (ej. Pinecone)

Diseñadas exclusivamente para almacenar vectores desde cero. Ofrecen baja latencia, escalabilidad en la nube y optimización precisa para algoritmos como HNSW.

2. Open Source / Auto-hospedadas (ej. Qdrant / Milvus)

Permiten un control total sobre la infraestructura y los datos. Ideales para entornos empresariales con estrictos requisitos de privacidad y seguridad.

3. Extensiones en DB Tradicionales (ej. pgvector en PostgreSQL)

Añaden capacidades de búsqueda vectorial sobre bases de datos SQL o NoSQL existentes, permitiendo combinar consultas relacionales y vectoriales en un solo lugar.

4. Integradas en Memoria / Locales (ej. Chroma / Faiss)

Librerías ligeras que corren localmente o en la memoria del servidor. Perfectas para prototipado rápido, desarrollo local y proyectos pequeños.

3. ARQUITECTURA GENERAL DEL FLUJO EN N8N

El flujo backend se construyó en la plataforma **n8n** integrando un Agente de Inteligencia Artificial que consulta Pinecone antes de generar una respuesta al frontend.



Figura 1: Diagrama completo del workflow en n8n (Webhook → AI Agent → Respond to Webhook)

4. CONFIGURACIÓN DETALLADA DE LOS NODOS N8N

Nodo 1: Webhook (Recepción de Peticiones)

Este nodo actúa como el punto de entrada HTTP POST desde el frontend. Se configura para escuchar peticiones en la ruta de producción `/webhook/pregunta` y redirigir el control al nodo *Respond to Webhook*.



Figura 2: Parámetros del nodo Webhook configurado con método POST.

Nodo 2: Simple Memory (Gestión de Contexto)

Mantiene el historial de la conversación conectándose directamente a la entrada de memoria del Agente de IA. Almacena las últimas 5 interacciones para proporcionar continuidad temática al usuario.

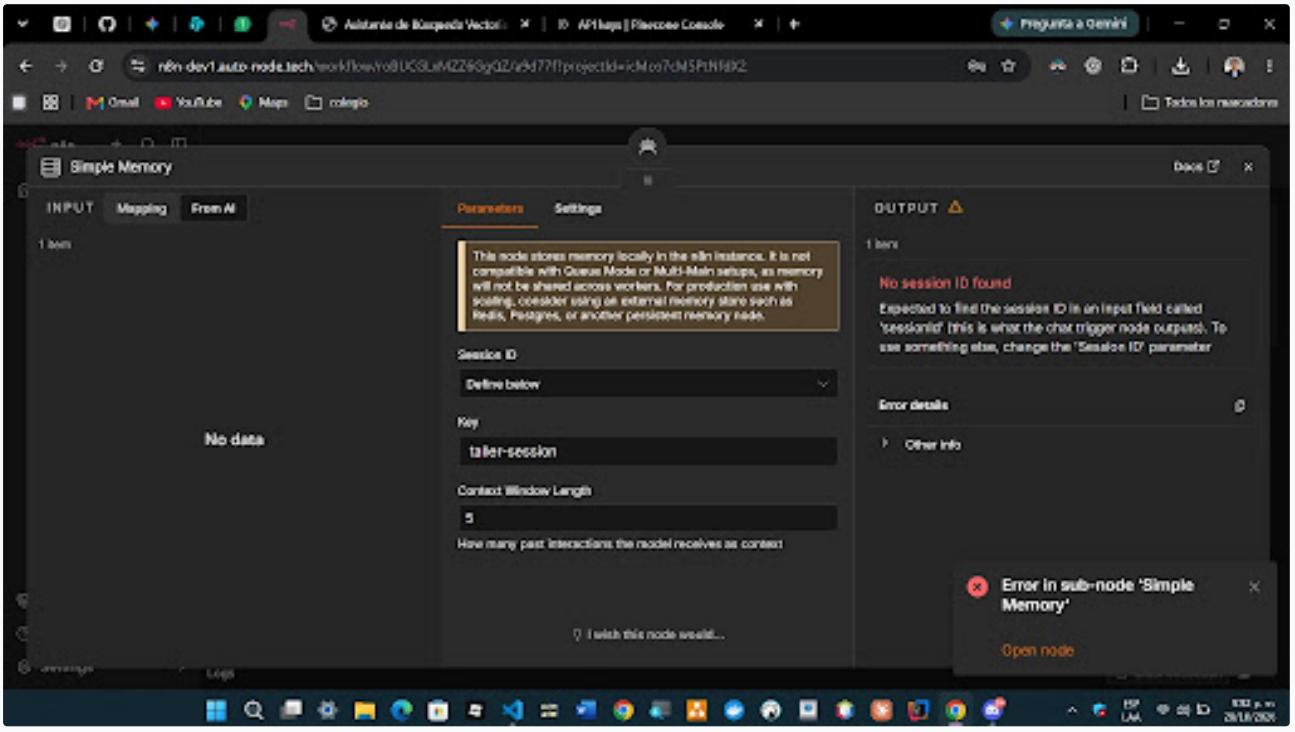


Figura 3: Configuración de la ventana de contexto y clave de sesión.

4. CONFIGURACIÓN DETALLADA DE LOS NODOS N8N (CONT.)

Nodo 3: OpenRouter Chat Model (Modelo LLM)

Se utilizó el modelo `nvidia/nemotron-3.5-lightning:free` a través de OpenRouter. Es el encargado de procesar la lógica de razonamiento y sintetizar la información devuelta por la base vectorial.

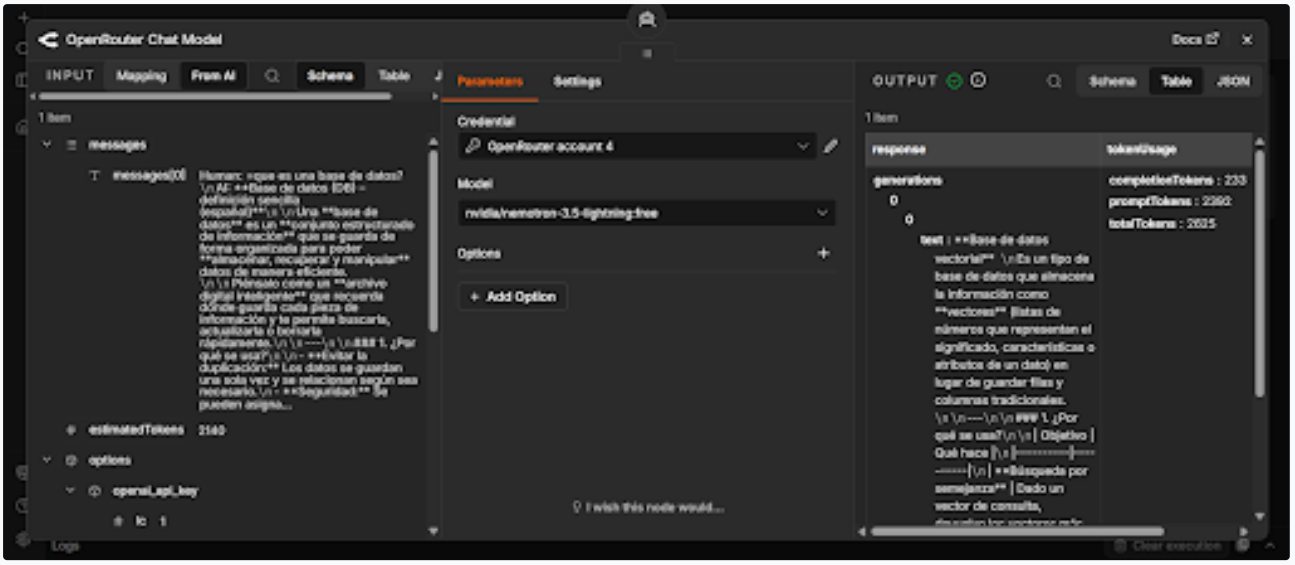


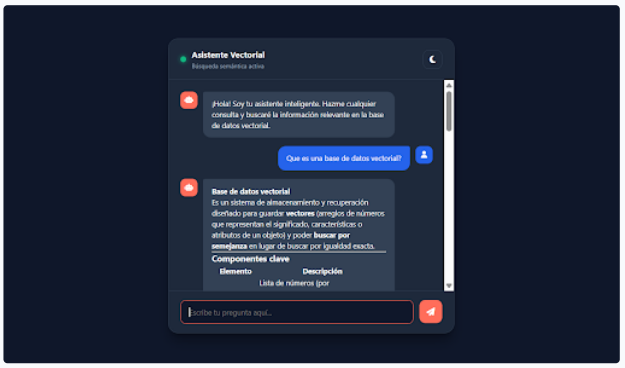
Figura 4: Configuración del modelo de lenguaje en OpenRouter.

Nodo 4: Pinecone Vector Store & Embeddings

Conectado como herramienta (*Tool*) al Agente de IA. Utiliza el modelo `sentence-transformers/all-MiniLM-L6-v2` de HuggingFace para convertir la consulta del usuario en vectores y recuperar los fragmentos relevantes indexados en Pinecone.



Pinecone Store



HuggingFace Embeddings

Nodo 5: Respond to Webhook (Respuesta Estructurada)

Envía la respuesta final al cliente web. Para evitar errores de sintaxis JSON provocados por saltos de línea e caracteres especiales de la IA, se configuró en modo **Expression** construyendo un objeto dinámico: `{{ { "output": $json.output } }}`.

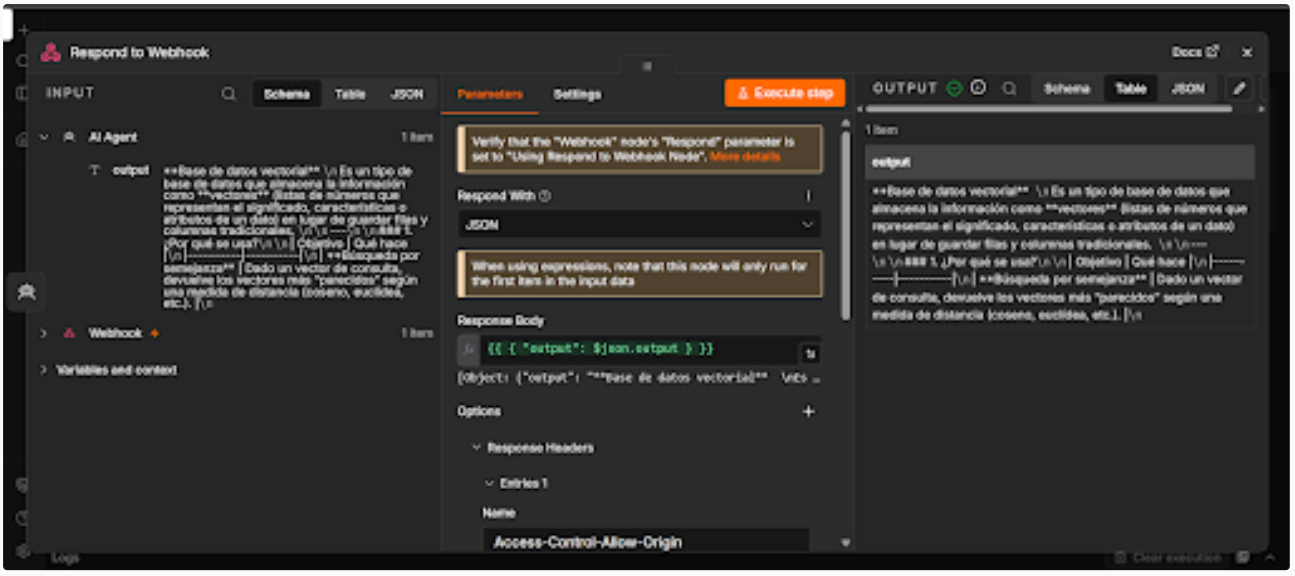


Figura 5: Estructura JSON del Response Body formateada correctamente.

5. MANEJO DEL FRONTEND E INTEGRACIÓN DE MARKED.JS

El frontend de la aplicación web administra la comunicación asíncrona mediante JavaScript nativo (`fetch`) enviando las preguntas del usuario al webhook de producción de n8n.

> **Soporte de Formato Markdown con marked.js:**
> Las respuestas generadas por los LLM contienen estructuras en sintaxis Markdown (como negritas `**texto**`, viñetas y tablas). Para evitar que se mostraran como texto plano con símbolos no procesados, se integró la librería ligera **marked.js**.
>
> En la función de renderizado (`appendMessage`), el mensaje emitido por el bot es procesado mediante `marked.parse(data.output)`, transformando dinámicamente las cadenas de texto en elementos HTML limpios y estructurados.

